SOS Emergency

API Request & Response Schemas

Schemas (JSON Schema draft 2020-12) for the GenUI emergency backend: a fast REST screen-composition endpoint, a voice health probe, and the live-voice WebSocket session. Field names align with the app's domain model (`A2uiNode`, `Classification`, `Severity` / `AppMode`) and the two-transport split in the technical specification.

Endpoints • GET /v1/voice/health • POST /v1/chat/stream • WS /v1/voice/agent

Generated 2026-06-26

Shared definitions ($defs)

Reusable subschemas referenced by the endpoint schemas below via `common.json#/$defs/<Name>`.


```
{
  "$id": "https://sos.local/schemas/common.json",
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$defs": {
    "Severity":     { "type": "string", "enum": ["neutral", "moderate", "high", "critical"] },
    "AppMode":      { "type": "string", "enum": ["triage", "crash", "medical", "threat", "roadsi
        de"] },
    "Connectivity": { "type": "string", "enum": ["online", "degraded", "offline"] },
    "CarState":     { "type": "string", "enum": ["driving", "parked"] },
    "ScenarioClass": {
      "type": "string",
      "enum": ["crash", "medical", "beingFollowed", "flatTire", "wontStart",
               "lockedOut", "outOfGas", "unsafeParked", "unknown"]
    },
    "Hazard": { "type": "string", "enum": ["smoke", "fire", "weather", "lowVisibility"] },

    "A2uiNode": {
      "type": "object",
      "required": ["type"],
      "properties": {
        "type":     { "type": "string", "description": "Catalog component name; unknown types re
            nder nothing." },
        "id":       { "type": "string", "description": "Optional stable id, used to merge stream
            ed chunks." },
        "props":    { "type": "object", "additionalProperties": true, "default": {} },
        "bindings": { "type": "object", "additionalProperties": { "type": "string" }, "default":
             {},
                      "description": "prop name -> DataModel path, e.g. \"/sos/countdown\"." },
        "children": { "type": "array", "items": { "$ref": "#/$defs/A2uiNode" }, "default": [] }
      },
      "additionalProperties": false
    },

    "Classification": {
      "type": "object",
      "required": ["scenario", "severity", "mode", "confidence"],
      "properties": {
        "scenario":   { "$ref": "#/$defs/ScenarioClass" },
        "severity":   { "$ref": "#/$defs/Severity" },
        "mode":       { "$ref": "#/$defs/AppMode" },
        "confidence": { "type": "number", "minimum": 0, "maximum": 1 },
        "hazards":    { "type": "array", "items": { "$ref": "#/$defs/Hazard" }, "default": [] }
      },
      "additionalProperties": false
    }
  }
}
```

1. GET /v1/voice/health

Liveness / readiness probe for the voice (and composition) backend. No request body; one optional query parameter.

Request — query parameters

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "VoiceHealthRequest",
  "type": "object",
  "properties": {
    "verbose": { "type": "boolean", "default": false,
                 "description": "Include per-dependency breakdown." }
  },
  "additionalProperties": false
}
```

Response — 200 OK (also returned with 503 body when status != ok)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "VoiceHealthResponse",
  "type": "object",
  "required": ["status", "version", "timestamp"],
  "properties": {
    "status":         { "type": "string", "enum": ["ok", "degraded", "down"] },
    "version":        { "type": "string", "examples": ["1.4.2"] },
    "uptimeSeconds":  { "type": "number", "minimum": 0 },
    "activeSessions": { "type": "integer", "minimum": 0 },
    "model": {
      "type": "object",
      "required": ["id", "ready"],
      "properties": {
        "id":    { "type": "string", "examples": ["claude-opus-4-8"] },
        "ready": { "type": "boolean" }
      }
    },
    "dependencies": {
      "type": "array",
      "description": "Present when verbose=true.",
      "items": {
        "type": "object",
        "required": ["name", "status"],
        "properties": {
          "name":      { "type": "string", "examples": ["asr", "tts", "llm"] },
          "status":    { "type": "string", "enum": ["ok", "degraded", "down"] },
          "latencyMs": { "type": "number", "minimum": 0 }
        }
      }
    },
    "timestamp": { "type": "string", "format": "date-time" }
  },
  "additionalProperties": false
}
```

Example response

```
{
  "status": "ok",
  "version": "1.4.2",
  "uptimeSeconds": 88112,
  "activeSessions": 3,
  "model": { "id": "claude-opus-4-8", "ready": true },
  "dependencies": [
    { "name": "asr", "status": "ok", "latencyMs": 41 },
    { "name": "tts", "status": "ok", "latencyMs": 63 },
    { "name": "llm", "status": "degraded", "latencyMs": 920 }
  ],
  "timestamp": "2026-06-26T17:04:22Z"
```

```
}
```

2. POST /v1/chat/stream

The fast screen-JSON composition endpoint (the `AiTransportRepository` channel). The client posts the deterministic `Classification` plus context and catalog version; the server returns the A2UI surface. Content is negotiated by `stream`: `text/event-stream` (SSE) when true, a single `application/json` body when false.

Request body

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ComposeRequest",
  "type": "object",
  "required": ["classification", "context", "catalogVersion"],
  "properties": {
    "classification": { "$ref": "common.json#/$defs/Classification" },
    "context": {
      "type": "object",
      "required": ["carState", "connectivity", "isNight", "locale"],
      "properties": {
        "carState":        { "$ref": "common.json#/$defs/CarState" },
        "connectivity":    { "$ref": "common.json#/$defs/Connectivity" },
        "isNight":         { "type": "boolean" },
        "locale":          { "type": "string", "examples": ["en-US"] },
        "emergencyNumber": { "type": "string", "examples": ["911", "112"] },
        "position": {
          "type": "object",
          "properties": {
            "latitude":  { "type": "number" },
            "longitude": { "type": "number" },
            "address":   { "type": "string" }
          }
        }
      }
    },
    "catalogVersion": { "type": "string", "examples": ["2026.06.0"],
                        "description": "Registry/manifest version the client can render." },
    "stream":         { "type": "boolean", "default": true },
    "timeBudgetMs":   { "type": "integer", "minimum": 100, "maximum": 10000, "default": 3000 },
    "requestId":      { "type": "string", "format": "uuid" }
  },
  "additionalProperties": false
}
```

Response (stream: false) — single JSON body

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ComposeResponse",
  "type": "object",
  "required": ["requestId", "surface"],
  "properties": {
    "requestId":      { "type": "string", "format": "uuid" },
    "catalogVersion": { "type": "string" },
    "surface":        { "$ref": "common.json#/$defs/A2uiNode",
                        "description": "Root is always an EmergencyRoot." },
    "finishReason":   { "type": "string", "enum": ["complete", "timeout", "refused"] },
    "usage": {
      "type": "object",
      "properties": {
        "latencyMs":    { "type": "number" },
        "inputTokens":  { "type": "integer" },
        "outputTokens": { "type": "integer" }
      }
    }
  },
  "additionalProperties": false
}
```

Response (stream: true) — text/event-stream; each data: line is one event

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ComposeStreamEvent",
  "oneOf": [
    {
      "type": "object",
      "required": ["type", "node"],
      "properties": {
        "type":     { "const": "node" },
        "parentId": { "type": "string", "description": "Merge target; null/absent = root." },
        "node":     { "$ref": "common.json#/$defs/A2uiNode" }
      },
      "additionalProperties": false
    },
    {
      "type": "object",
      "required": ["type", "finishReason"],
      "properties": {
        "type":         { "const": "done" },
        "finishReason": { "type": "string", "enum": ["complete", "timeout", "refused"] }
      },
      "additionalProperties": false
    },
    {
      "type": "object",
      "required": ["type", "code", "message"],
      "properties": {
        "type":    { "const": "error" },
        "code":    { "type": "string", "examples": ["invalid_layout", "model_unavailable"] },
        "message": { "type": "string" }
      },
      "additionalProperties": false
    }
  ]
}
```

Example SSE stream

```
data: {"type":"node","node":{"type":"EmergencyRoot","props":{"tier":"high","carState":"driving"}
    }}

data: {"type":"node","parentId":"root","node":{"type":"SeverityBanner","props":{"tier":"high","l
    abel":"High","context":"Act soon"}}}

data: {"type":"node","parentId":"root","node":{"type":"GuidanceCallout","props":{"tier":"high","
    kicker":"Do this now","text":"Drive to the nearest police station"}}}

data: {"type":"done","finishReason":"complete"}
```

The client time-boxes this call and falls back to the deterministic baseline on timeout / error / invalid layout. The Safety Supervisor validates the surface before it renders, and the persistent 911 + share-location rail is part of EmergencyRoot — never the AI's composition — so it can never be removed.

3. WS /v1/voice/agent

Full-duplex live voice session (the `VoiceSessionRepository` channel). After the socket opens, both sides exchange JSON text frames discriminated by `type`. Audio is base64 in the control channel here; a binary-frame variant can carry raw Opus/PCM instead.

Client → Server (request frames)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "VoiceClientFrame",
  "oneOf": [
    {
      "type": "object",
      "required": ["type", "config"],
      "properties": {
        "type": { "const": "session.start" },
        "config": {
          "type": "object",
          "required": ["locale", "sampleRate", "codec"],
          "properties": {
            "locale":     { "type": "string", "examples": ["en-US"] },
            "sampleRate": { "type": "integer", "enum": [8000, 16000, 24000, 48000] },
            "codec":      { "type": "string", "enum": ["pcm16", "opus"] },
            "incidentId": { "type": "string", "format": "uuid" },
            "tier":       { "$ref": "common.json#/$defs/Severity" }
          }
        }
      },
      "additionalProperties": false
    },
    {
      "type": "object",
      "required": ["type", "seq", "audio"],
      "properties": {
        "type":  { "const": "audio.chunk" },
        "seq":   { "type": "integer", "minimum": 0 },
        "audio": { "type": "string", "contentEncoding": "base64",
                   "description": "One frame of mic audio in the negotiated codec." }
      },
      "additionalProperties": false
    },
    { "type": "object", "required": ["type"],
      "properties": { "type": { "const": "audio.commit" } },
      "description": "End of the current user utterance (VAD or push-to-talk release)." },
    {
      "type": "object",
      "required": ["type", "text"],
      "properties": { "type": { "const": "text" }, "text": { "type": "string" } },
      "description": "Typed input fallback."
    },
    {
      "type": "object",
      "required": ["type", "action"],
      "properties": {
        "type":   { "const": "control" },
        "action": { "type": "string", "enum": ["mute", "unmute", "barge_in", "cancel"] }
      }
    },
    { "type": "object", "required": ["type"],
      "properties": { "type": { "const": "session.end" } } }
  ]
}
```

Server → Client (response frames)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "VoiceServerFrame",
```

```
"oneOf": [
  {
    "type": "object",
    "required": ["type", "sessionId"],
    "properties": {
      "type":      { "const": "session.ready" },
      "sessionId": { "type": "string", "format": "uuid" },
      "config": {
        "type": "object",
        "properties": {
          "sampleRate": { "type": "integer" },
          "codec":      { "type": "string", "enum": ["pcm16", "opus"] }
        }
      }
    }
  },
  {
    "type": "object",
    "required": ["type", "role", "text", "isFinal"],
    "properties": {
      "type":    { "const": "transcript" },
      "role":    { "type": "string", "enum": ["user", "agent"] },
      "text":    { "type": "string" },
      "isFinal": { "type": "boolean" }
    }
  },
  {
    "type": "object",
    "required": ["type", "seq", "audio"],
    "properties": {
      "type":  { "const": "audio.chunk" },
      "seq":   { "type": "integer", "minimum": 0 },
      "audio": { "type": "string", "contentEncoding": "base64",
                 "description": "Agent TTS audio in the negotiated codec." }
    }
  },
  {
    "type": "object",
    "required": ["type", "intent", "confidence"],
    "properties": {
      "type":       { "const": "intent" },
      "intent":     { "type": "string",
                      "enum": ["carProblem", "crash", "medical", "feelUnsafe", "beingFollowed"
                          ,
                               "lockedOut", "roadside", "outOfGas", "document", "none"] },
      "confidence": { "type": "number", "minimum": 0, "maximum": 1 },
      "slots":      { "type": "object", "additionalProperties": true }
    },
    "description": "Surfaced intent fed back into the orchestrator loop as a UserSignal."
  },
  {
    "type": "object",
    "required": ["type", "name"],
    "properties": {
      "type": { "const": "event" },
      "name": { "type": "string", "enum": ["vad_start", "vad_stop", "barge_in", "thinking"] }
    }
  },
  {
    "type": "object",
    "required": ["type", "code", "message"],
    "properties": {
      "type":    { "const": "error" },
      "code":    { "type": "string", "examples": ["unauthorized", "codec_unsupported"] },
      "message": { "type": "string" },
      "fatal":   { "type": "boolean", "default": false }
    }
  },
  {
    "type": "object",
    "required": ["type", "reason"],
    "properties": {
```

```
        "type":   { "const": "session.closed" },
        "reason": { "type": "string", "enum": ["client_request", "timeout", "completed", "error"
            ] }
      }
    }
  ]
}
```

Example exchange

```
-> {"type":"session.start","config":{"locale":"en-US","sampleRate":16000,"codec":"opus","tier":"
    critical"}}
<- {"type":"session.ready","sessionId":"7b3...","config":{"sampleRate":16000,"codec":"opus"}}
-> {"type":"audio.chunk","seq":0,"audio":"T2dnUw..."}
-> {"type":"audio.commit"}
<- {"type":"transcript","role":"user","text":"there's smoke from the hood","isFinal":true}
<- {"type":"intent","intent":"carProblem","confidence":0.91,"slots":{"hazard":"smoke"}}
<- {"type":"transcript","role":"agent","text":"Pull over and turn off the engine.","isFinal":tru
    e}
<- {"type":"audio.chunk","seq":0,"audio":"T2dnUw..."}
```

The REST composition channel (/v1/chat/stream) and the WebSocket voice channel (/v1/voice/agent) are independent and fail independently: a dropped voice socket never blocks screen composition, and a slow/failed compose call never interrupts an active voice session. Voice intents re-enter the deterministic loop as UserSignals, which may trigger a fresh compose call.